Analytical Problem Solving

Coordinate convention: A cell is written as (row, column). The robot starts at (1,1) and must reach (5,5). Blocked cells (shelves) are (2,2), (2,3), (3,3), (4,2) and (4,4). Each legal move costs 1.

A. PEAS Description

PEAS element	Description
Performance measure	Get the package to (5,5) without hitting shelves or leaving the grid, using as few moves as possible. Since each move costs 1, a shorter route has a lower cost.
Environment	A 5 x 5 warehouse grid. Start at (1,1), goal at (5,5). Shelves at (2,2), (2,3), (3,3), (4,2), (4,4).
Actuators	Wheels or motors for one-cell moves (Up, Down, Left, Right) and a gripper to carry and drop the package.
Sensors	Position sensor for the current cell; map or obstacle sensor for shelves; package and destination sensors to confirm delivery.

B. Best-Suited Agent Type

A **goal-based agent** fits best. The robot has one clear goal, reaching (5,5) with the package, and can plan a route to it using its map of the grid and the shelves.

A simple reflex agent reacts only to what is next to it and could fail to find a complete route. A utility-based agent is only needed when objectives compete (time, energy, risk). Here there is a single goal and every move costs the same, so goal-based planning is appropriate.

C. Search Problem Formulation

• **State:** The robot's current cell (row, column).

• **Initial state:** (1,1).

• **Goal state:** (5,5).

• **Actions:** Up, Down, Left, Right.

• **Transition model:** An action moves the robot one cell in that direction if the target cell is inside the grid and is not blocked. Otherwise the move is illegal. For example, Right from (r,c) leads to (r,c+1).

• **Path cost:** Each move costs 1, so a path of k moves has cost g(path) = k.

D. Breadth-First Search (BFS)

BFS uses a FIFO queue, so the earliest state added is expanded first. It explores paths in order of length, and with equal move costs it finds a shortest path. Neighbours are checked in the order Up, Down, Left, Right, and a cell is marked discovered when it enters the queue.

Step	Node expanded	Queue after expansion
1	(1,1)	(2,1), (1,2)

2	(2,1)	(1,2), (3,1)
3	(1,2)	(3,1), (1,3)
4	(3,1)	(1,3), (4,1), (3,2)

Step	Node expanded	Queue after expansion
5	(1,3)	(4,1), (3,2), (1,4)
6	(4,1)	(3,2), (1,4), (5,1)
7	(3,2)	(1,4), (5,1)
8	(1,4)	(5,1), (2,4), (1,5)
9	(5,1)	(2,4), (1,5), (5,2)
10	(2,4)	(1,5), (5,2), (3,4), (2,5)
11	(1,5)	(5,2), (3,4), (2,5)
12	(5,2)	(3,4), (2,5), (5,3)
13	(3,4)	(2,5), (5,3), (3,5)
14	(2,5)	(5,3), (3,5)
15	(5,3)	(3,5), (4,3), (5,4)
16	(3,5)	(4,3), (5,4), (4,5)
17	(4,3)	(5,4), (4,5)
18	(5,4)	(4,5), (5,5)
19	(4,5)	(5,5)
20	(5,5)	Goal reached; queue empty

BFS path: (1,1) -> (2,1) -> (3,1) -> (4,1) -> (5,1) -> (5,2) -> (5,3) -> (5,4) -> (5,5)

Actions: Down, Down, Down, Down, Right, Right, Right, Right

Total cost: 8 x 1 = **8**

E. Depth-First Search (DFS)

DFS uses a LIFO stack, so the most recently added state is expanded first. To explore neighbours in the order Up, Down, Left, Right, they are pushed in reverse. The top of the stack is on the left, and a cell is marked discovered when it is pushed.

Step	Node expanded	Stack after expansion (top first)

1	(1,1)	(2,1), (1,2)
2	(2,1)	(3,1), (1,2)
3	(3,1)	(4,1), (3,2), (1,2)
4	(4,1)	(5,1), (3,2), (1,2)
5	(5,1)	(5,2), (3,2), (1,2)
6	(5,2)	(5,3), (3,2), (1,2)
7	(5,3)	(4,3), (5,4), (3,2), (1,2)
8	(4,3)	(5,4), (3,2), (1,2)
9	(5,4)	(5,5), (3,2), (1,2)
10	(5,5)	Goal reached (remaining on stack: (3,2), (1,2))

DFS path: (1,1) -> (2,1) -> (3,1) -> (4,1) -> (5,1) -> (5,2) -> (5,3) -> (5,4) -> (5,5)

Actions: Down, Down, Down, Down, Right, Right, Right, Right

Total cost: 8 x 1 = **8**

DFS gives the same path as BFS here because of this grid and neighbour order. In general, DFS is not guaranteed to find the shortest path.

F. BFS and DFS Comparison

Criterion	**BFS**	**DFS**
Completeness	Complete on this finite grid when repeated states are avoided.	Complete on this finite grid when repeated states are avoided.
Optimality	Finds a shortest path because every move costs 1.	Not generally optimal; returns the first route it finds.
Time complexity	$O(b^d)$, where b is the branching factor and d is the depth of the nearest goal.	$O(b^m)$, where m is the maximum search depth.
Space complexity	$O(b^d)$; may store many frontier states.	O(bm); usually stores less of the frontier.
Suitability here	Finds a shortest route, matching the requirement to minimize moves.	Can use less memory, but does not guarantee a shortest route.

Optional Python code:

```
from collections import deque

SIZE = 5
START = (1, 1)
GOAL = (5, 5)
```

```
BLOCKED = {(2, 2), (2, 3), (3, 3), (4, 2), (4, 4)}

# Neighbour order used in the written BFS and DFS traces.
MOVES = [
 ("Up", (-1, 0)),
 ("Down", (1, 0)),
 ("Left", (0, -1)),
 ("Right", (0, 1)),
]

def neighbors(cell):
 row, col = cell
 for action, (dr, dc) in MOVES:
 nxt = (row + dr, col + dc)
 inside = 1 <= nxt[0] <= SIZE and 1 <= nxt[1] <= SIZE  if
inside and nxt not in BLOCKED:
 yield action, nxt

def reconstruct_path(parent, state):
 path = []
 while state in parent:
 prev, action = parent[state]
 path.append((action, state))
 state = prev
 return list(reversed(path))

def bfs():
 frontier = deque([START])
 discovered = {START}
 parent = {}
 while frontier:
 current = frontier.popleft()
 if current == GOAL:
 return reconstruct_path(parent, GOAL)  for
action, nxt in neighbors(current):
 if nxt not in discovered:
 discovered.add(nxt)
 parent[nxt] = (current, action)
frontier.append(nxt)
 return None

def dfs():
 stack = [START]
 discovered = {START}
 parent = {}
 while stack:
 current = stack.pop()
 if current == GOAL:
 return reconstruct_path(parent, GOAL)  # Reverse the order
so Up is explored first.  for action, nxt in
reversed(list(neighbors(current))):  if nxt not in
discovered:
 discovered.add(nxt)
 parent[nxt] = (current, action)
stack.append(nxt)
 return None

for name, search in [("BFS", bfs), ("DFS",
dfs)]:  path = search()
 print(f"\n{name}")
 if path is None:
 print("No path found.")
 else:
 print("Start:", START)
 for action, cell in path:
 print(f"{action} -> {cell}")
 print("Total cost:", len(path))
```

OUTPUT:

```
BFS
Start: (1, 1)
Down -> (2, 1)
Down -> (3, 1)
Down -> (4, 1)
Down -> (5, 1)
```

```
Right -> (5, 2)
Right -> (5, 3)
Right -> (5, 4)
Right -> (5, 5)
Total cost: 8

DFS
Start: (1, 1)
Down -> (2, 1)
Down -> (3, 1)
Down -> (4, 1)
Down -> (5, 1)
Right -> (5, 2)
Right -> (5, 3)
Right -> (5, 4)
Right -> (5, 5)
Total cost: 8
```

Conclusion:

BFS is the better choice when the robot must use the fewest moves. DFS may need less memory, but BFS guarantees a shortest path in this problem because all moves have equal cost.